

.NET Platform & Dapper Guide

Evolution, Core Architecture, Packages, Models & Micro-ORM Integration

.NET is a free, cross-platform, open-source developer framework built by Microsoft for constructing web applications, mobile apps, desktop solutions, cloud services, and IoT software.

1. Evolution & Version History

- .NET Framework (2002 – 2019, v1.0 to v4.8):** The legacy, Windows-only framework. It relied heavily on Windows APIs, IIS (Internet Information Services), and global assembly caches.
- .NET Core (2016 – 2019, v1.0 to v3.1):** A complete rewrite built from scratch to be modular, high-performance, open-source, and fully cross-platform (running natively on Linux, macOS, and Windows).
- Unified .NET (.NET 5 to .NET 8+):** Microsoft dropped the "Core" branding starting with .NET 5 to unify .NET Framework, .NET Core, and Xamarin/Mono into a single platform.
- Release Cadence:** Microsoft maintains a predictable release schedule where even-numbered versions (e.g., .NET 6, .NET 8) receive Long-Term Support (LTS) for 3 years, while odd-numbered versions (e.g., .NET 7, .NET 9) are Standard Term Support (STS) releases receiving updates for 18 months.

2. Core Concepts: Packages, Models, and Architecture

Concept	Explanation
NuGet Packages	The official package manager for .NET. Libraries, dependencies, and tools are distributed as <code>.nupkg</code> files via public (nuget.org) or private feeds. You install them via the CLI (<code>dotnet add package <PackageName></code>) or Visual Studio.
Models	Plain Old CLR Objects (POCOs) or C# <code>record</code> types that represent domain entities, data schemas, or Data Transfer Objects (DTOs) used across controller endpoints and database operations.
CLR (Common Language Runtime)	The execution engine of .NET. It provides memory management (Garbage Collection), type safety, exception handling, thread execution, and Just-In-Time (JIT) compilation from Intermediate Language (IL) to native machine code.
CLI (dotnet command)	A cross-platform toolchain used to build, package, test, restore packages, and run .NET applications directly from the command line.

3. What is Dapper?

Dapper is a lightweight, open-source Object-Relational Mapper (ORM) — often classified as a Micro-ORM — created by Stack Overflow to solve performance bottlenecks associated with traditional heavy ORMs.

Key Features of Dapper:

- Speed & Performance:** Dapper executes raw SQL queries and maps the query results directly to C# object models almost as fast as using native `SqlCommand` / `SqlDataReader` execution loops.
- Extension Methods:** Dapper extends the standard ADO.NET interface (`IDbConnection`), allowing you to execute queries directly from any connection object (`SqlConnection` , `NpgsqlConnection` , `MySqlConnection` , `SqliteConnection`).

- **Raw SQL Control:** Unlike full-featured ORMs like Entity Framework (EF) Core, Dapper does not generate SQL dynamically for complex queries, maintain a change tracker, or auto-generate schema migrations. You write explicit raw SQL statements.

Example Usage:

```
using System.Data;
using Microsoft.Data.SqlClient;
using Dapper;

public record UserDto(int Id, string Name, string Email);

public async Task<UserDto?> GetUserByIdAsync(string connectionString, int userId)
{
    using IDbConnection db = new SqlConnection(connectionString);

    // Dapper extends IDbConnection with QueryFirstOrDefaultAsync
    string sql = "SELECT Id, Name, Email FROM Users WHERE Id = @Id";

    return await db.QueryFirstOrDefaultAsync<UserDto>(sql, new { Id = userId });
}
```

Dapper vs. Entity Framework (EF Core):

- **Use Dapper when:** High throughput and low latency are critical, complex SQL queries/stored procedures are already written, or memory overhead needs to be kept to an absolute minimum.
- **Use EF Core when:** You need full-fledged LINQ query generation, automatic entity state tracking, database migration management, and complex object relationship mappings without writing hand-crafted SQL.